# GenAI-Enhanced Advanced Data Structures

Student Name:Jiawei Li
Student ID: 25212461
Data Structure: Segment Tree (with lazy propagation)
Scenario: Daily retail sales analytics

**Data Structure Design**

I chose a Segment Tree because I wanted to learn a data structure I had not implemented before, and lazy propagation struck me as a technique worth understanding properly. Once the data structure was chosen, I looked for a scenario that would exercise all its operations naturally. Retail sales analytics was a strong fit: a chain records daily revenue across a fixed window, and the analytics engine needs range-sum queries for revenue reports, point updates for POS corrections, and range-add for bulk adjustments such as a supplier rebate backdated across a month. These map directly to ‘rangeSum’, ‘pointUpdate’, and ‘rangeAdd’.

The key design decision was to include lazy propagation. Without it, a range update costs O((r − l) · log n) by touching every affected leaf. Lazy propagation lets a fully-contained ancestor record a pending delta in a parallel ‘lazy[]’ array, pushing it down only when a later operation descends through that node. This brings both ‘rangeAdd’ and ‘rangeSum’ to O(log n). My benchmark confirms it: at n = 100,000 the segment tree is over 17× faster than the naive baseline.

I adopted the convention that ‘tree[node]’ is always the current aggregate while ‘lazy[node]’ stores pending deltas for children only. The alternative — stale ‘tree[node]’ until lazy is applied — complicates every query path unnecessarily. The tree uses a 1-indexed array of size 4n, sidestepping non-power-of-two edge cases, and midpoints are computed with ‘(start + end) >>> 1’ to prevent the classic signed-overflow bug.

Structurally, I separated ‘SegmentTree’ (a plain integer-indexed data structure) from ‘SalesAnalytics’ (a thin domain facade translating dates and monetary amounts into indices). This keeps the core algorithm reusable. Monetary values are stored as ‘long’ cents rather than ‘double’ euros, eliminating floating-point drift and keeping the differential test deterministic. The constructor rejects empty input explicitly, since no public method is meaningful on an empty tree.

I rejected two alternatives: a generic ‘SegmentTree<T, BinaryOperator<T>>‘ (adds boilerplate without benefit) and a Fenwick Tree (does not natively support O(log n) range updates without pairing two BITs).

**Use of Generative AI**

I used three GenAI tools, each in a distinct role. All interactions are documented in ‘genai_log.md’.

ChatGPT (GPT-5.4) was my starting point for understanding and planning. I fed it the brief and asked for a concise deliverables checklist, which helped me break the assignment into manageable steps. I then asked it to suggest real-world scenarios where a Segment Tree would be a natural fit; it proposed retail sales analytics, sensor monitoring, and game leaderboards. I chose retail sales because the operation mix, range-sum queries, single-day corrections, bulk adjustments, maps directly onto the tree's API. Later in the project, ChatGPT also suggested optimisations: the unsigned-shift midpoint to avoid overflow, and ‘long’ cents instead of ‘double’ euros to keep the differential test deterministic. I accepted both, though the cents-representation idea was something I had already been leaning towards.

Claude (Opus 4.6) was my primary coding partner. I used it to produce initial drafts of ‘SegmentTree.java’, ‘SalesAnalytics.java’, the test harness, and the benchmark. It also served as an algorithm explainer: when I was unsure about the ‘push()’ invariant, I asked it to walk through a concrete example of lazy propagation across a partially overlapping query, which made the convention click for me. On the debugging side, an earlier draft of ‘push()’ double-counted the delta by applying it to ‘tree[node]’ a second time before pushing to children, a subtle bug. My randomised differential test (5,000 operations cross-checked against a naive baseline) caught it immediately; I brought the failing output back to Claude, which helped trace the root cause and produce a correct rewrite. This was the single most instructive interaction of the project: it showed that AI-generated code can be plausible-looking yet subtly wrong, and that automated tests are the essential safety net.

Google Gemini served as a second-opinion reviewer. It confirmed the lazy logic was sound but suggested switching to a minimum-tree, a misread of my prompt, since the domain is additive. I rejected it outright. It also recommended JMH for benchmarking, which I declined as overkill; I kept a hand-rolled benchmark with a warm-up pass and a JIT-defeating blackhole pattern instead.

What worked well: ChatGPT turned a vague brief into a concrete plan; Claude was precise and productive for code generation and debugging. What did not: the "confidently wrong" ‘push()’ bug is genuinely dangerous for code with subtle invariants, and the tendency to over-engineer (generic types, JMH, Maven) required restraint. Human judgement was

essential for the scenario choice, the data-structure / domain-wrapper boundary, the test strategy, and the cents-not-doubles decision, none suggested unprompted by any tool.

**Critical Reflection**

GenAI genuinely improved my understanding of lazy propagation. Before this project I knew the concept abstractly but had not implemented ‘push()’ from scratch. Claude's worked example clarified why ‘tree[node]’ must already reflect the lazy delta while ‘lazy[node]’ is pending for children, and ChatGPT's checklist approach helped me see the assignment as concrete deliverables rather than one monolithic task. That said, I only trusted my understanding after re-deriving the push logic by hand and validating it against 5,000 random operations. The AI gave me a starting point; the tests gave me confidence.

The clearest risk was the "confidently wrong" failure mode. The buggy ‘push()’ looked entirely plausible, clean code, good naming, correct structure. But it double-counted the delta. Without a randomised differential test I might never have caught it by inspection alone. This is not a hypothetical concern: lazy propagation is exactly the kind of subtle invariant where a small mistake produces outputs that look reasonable on small inputs but diverge at scale. My test harness (1,897 assertions, 0 failures) was the single most important quality gate in the project.

A second risk was over-engineering. AI tools consistently suggested adding complexity, generic types, JMH, Maven that would have increased submission friction without improving the deliverable. Learning to say "no" to a plausible suggestion requires understanding the context of the task, which AI does not provide.

Going forward I would continue using GenAI for scaffolding, explanation, and code review, but never trust it without strong automated tests for code involving subtle invariants which are lazy propagation, lock-free concurrency, numerical stability. The running ‘genai_log.md’ also proved valuable for attribution hygiene: it is easy to accept an AI suggestion verbatim and forget it was AI-authored, and the log helped me stay honest about what was mine and what was not.

**AI Usage Declaration**

I used the following Generative AI tools while completing this assignment:

- ChatGPT (GPT-5.4) — used for understanding the assignment brief, planning a task checklist, generating real-world scenario ideas for the Segment Tree, and suggesting optimisations. Representative prompts and decisions are documented in ‘genai_log.md’.

- Claude (Opus 4.6) — used for producing draft code snippets, explaining algorithms (especially lazy propagation mechanics), and identifying bugs and performance issues.

- Google Gemini — used for suggesting optimisations and as a second-opinion code reviewer for the lazy propagation logic.

All final design decisions, algorithmic reasoning, test strategy, and report prose are my own. Where I have reused or adapted AI-produced code, I have reviewed and modified it and kept a record of the changes in ‘genai_log.md’. I accept full responsibility for the correctness of the submitted code.